

VLSI Design of FIR Filter for DSP Applications

Abstract

This project presents the VLSI design and implementation of a Finite Impulse Response (FIR) filter for Digital Signal Processing (DSP) applications. FIR filters are widely used in signal processing because of their stability and linear phase characteristics. In this project, filter coefficients were generated using GNU Octave/MATLAB. The FIR filter was implemented using Verilog HDL and simulated in Xilinx Vivado Design Suite. Synthesis was performed, and the results were analyzed for area, speed, and power. The output waveforms confirmed correct operation of the filter. The design demonstrates efficient implementation of FIR filters for DSP applications using VLSI techniques.

1. Introduction

Digital Signal Processing is an important area in communication and embedded systems. FIR filters are commonly used to remove unwanted frequency components and improve signal quality.

A Finite Impulse Response filter is defined by:

$$y[n] = \sum h[k]x[n-k]$$

Where:

- $y[n]$ = output signal
- $x[n]$ = input signal
- $h[k]$ = filter coefficients

Advantages of the FIR filter:

- Stable operation

- Linear phase response
- Easy hardware implementation
- Suitable for VLSI systems

Applications:

- Audio processing
- Image processing
- Biomedical signal filtering
- Wireless communication

2. Objective

- To design FIR filter coefficients using GNU Octave/MATLAB
- To implement an FIR filter using Verilog HDL
- To simulate and synthesize using Vivado
- To compare area, speed, and power performance

3. Software Tools Used

Tool	Purpose
GNU Octave / MATLAB	Coefficient generation
Xilinx Vivado 2025.2	Verilog simulation and synthesis
Verilog HDL	Hardware design

4. OCTAVE CODE:

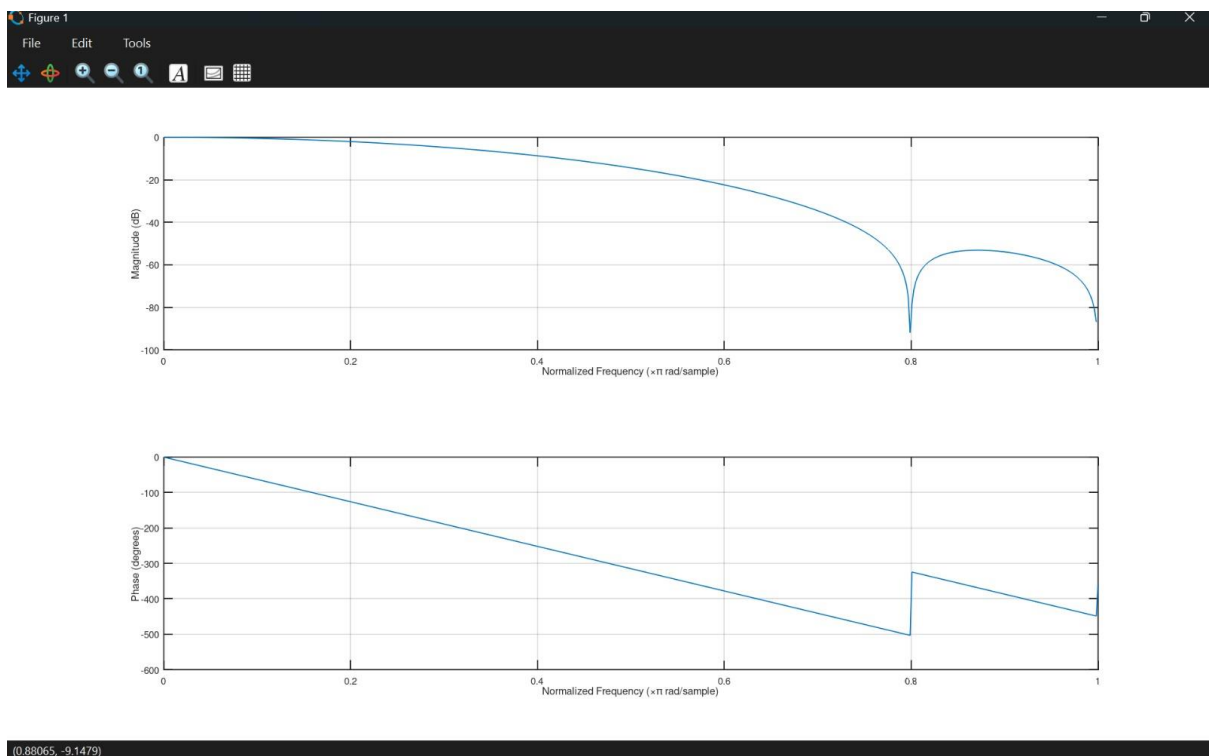
```
pkg load signal
```

```
N = 8;
```

```
fc = 0.3;
```

```
b = fir1(N-1, fc);  
disp('FIR coefficients:')  
disp(b)  
freqz(b,1)  
grid on
```

5. OCTAVE OUTPUT GRAPH & COEFFICIENTS:



```
Command Window
GNU Octave (x86_64-mingw32) version 11.1.0
Copyright (C) 1993-2026 The Octave Project Developers
License GPLv3+: GNU GPL version 3 or later <https://g

This is free software; see the source code for copyp
There is NO WARRANTY, to the extent permitted by law.

Home page: <https://octave.org>
Support resources: <https://octave.org/support>
Improve Octave: <https://octave.org/get-involve

For changes from previous versions, type 'news'.

octave:1> pkg load signal
octave:2> N = 8;
octave:3> fc = 0.3;
octave:4> b = fir1(N-1, fc);
octave:5> disp('FIR coefficients:')
FIR coefficients:
octave:6> disp(b)
-1.2294e-03  2.6618e-02  1.5590e-01  3.1871e-01
octave:7> freqz(b,1)
octave:8> grid on
```

6.VIVADO CODE:

A. Fir_filter_v:

```
module fir_filter(
    input clk,
    input reset,
    input signed [7:0] x,
    output reg signed [15:0] y
);

reg signed [7:0] delay[0:7];

always @(posedge clk) begin
    if(reset) begin
```

```

    delay[0] <= 0;
    delay[1] <= 0;
    delay[2] <= 0;
    delay[3] <= 0;
    delay[4] <= 0;
    delay[5] <= 0;
    delay[6] <= 0;
    delay[7] <= 0;
    y <= 0;
end
else begin
    delay[7] <= delay[6];
    delay[6] <= delay[5];
    delay[5] <= delay[4];
    delay[4] <= delay[3];
    delay[3] <= delay[2];
    delay[2] <= delay[1];
    delay[1] <= delay[0];
    delay[0] <= x;

    y <=
        delay[0]*0 +
        delay[1]*7 +
        delay[2]*40 +
        delay[3]*82 +
        delay[4]*82 +
        delay[5]*40 +
        delay[6]*7 +
        delay[7]*0;

```

```
        end
    end

endmodule
```

B. fir_filter_tb:

```
`timescale 1ns/1ps
module fir_filter_tb;
    reg clk;
    reg reset;
    reg signed [7:0] x;
    wire signed [15:0] y;

    fir_filter uut(
        .clk(clk),
        .reset(reset),
        .x(x),
        .y(y)
    );

    always #5 clk = ~clk;

    initial begin
        clk = 0;
        reset = 1;
        x = 0;

        #10 reset = 0;
```

```
x = 10; #10;
x = 20; #10;
x = 30; #10;
x = 40; #10;
x = 50; #10;
x = 0; #50;

$stop;

end

endmodule
```

7. FIR Filter Design

Filter Specifications

- Filter Type: Low Pass FIR
- Number of Taps: 8
- Cutoff Frequency: 0.3

Coefficients Generated

```
h0 = -1.2294e-03
h1 = 2.6618e-02
h2 = 1.5590e-01
h3 = 3.1871e-01
h4 = 3.1871e-01
h5 = 1.5590e-01
h6 = 2.6618e-02
h7 = -1.2294e-03
```

Integer coefficients used in Verilog

```
0, 7, 40, 82, 82, 40, 7, 0
```

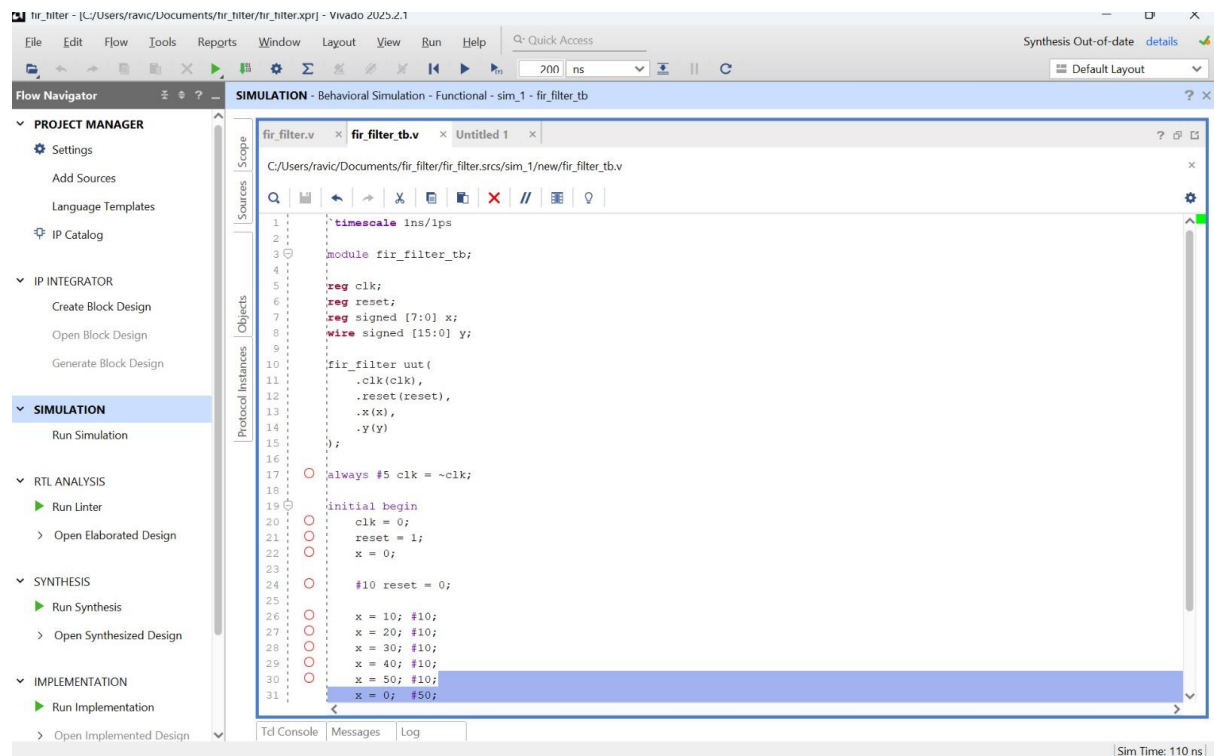
8. Verilog Implementation

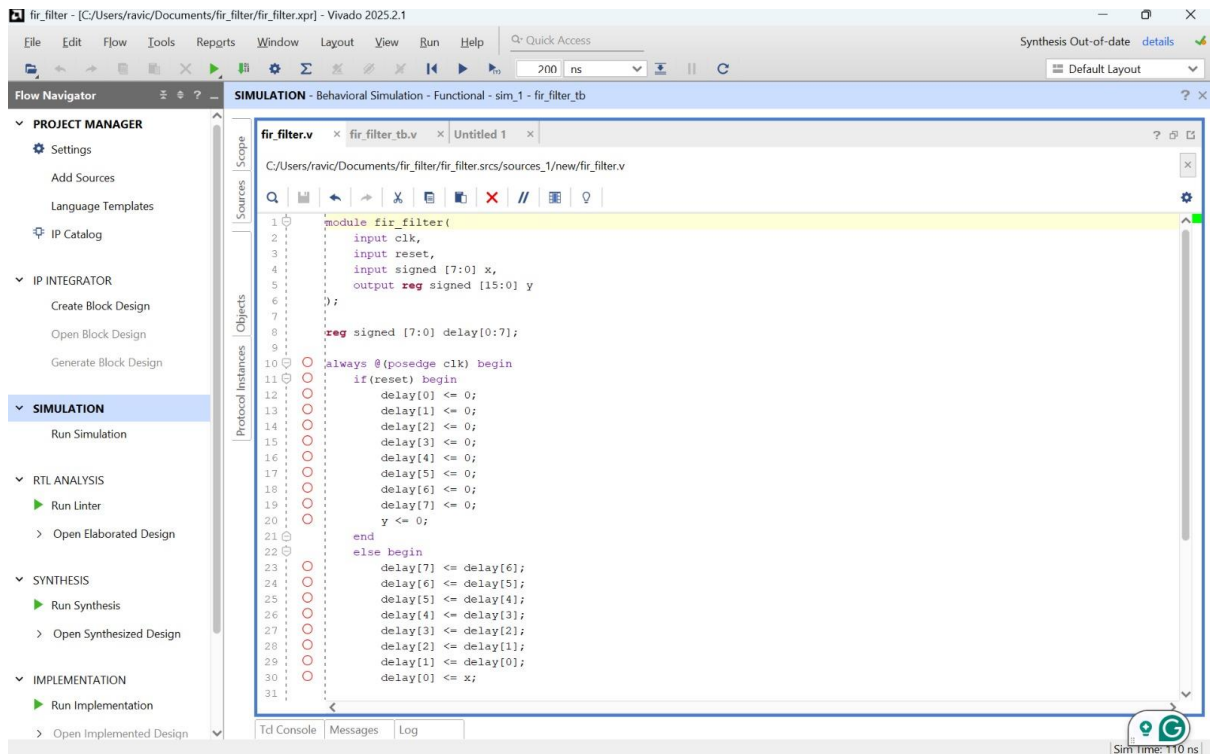
The FIR filter was implemented using:

- Delay registers
- Multipliers
- Adders
- Output register

Working:

1. Input signal applied through the testbench
2. Samples stored in shift registers
3. Each sample multiplied by the coefficient
4. All products added
5. Final output produced





9. Simulation

The simulation was carried out using Xilinx Vivado.

Test inputs applied:

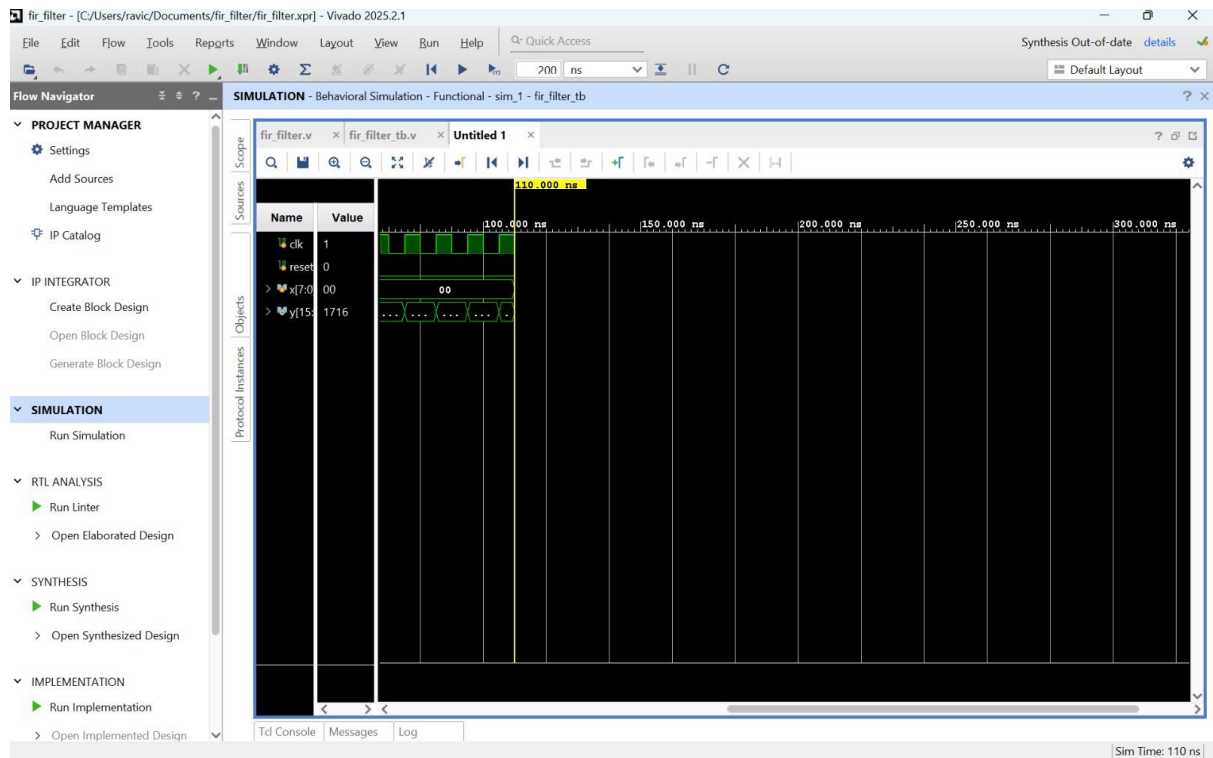
0, 10, 20, 30, 40, 50

Observed:

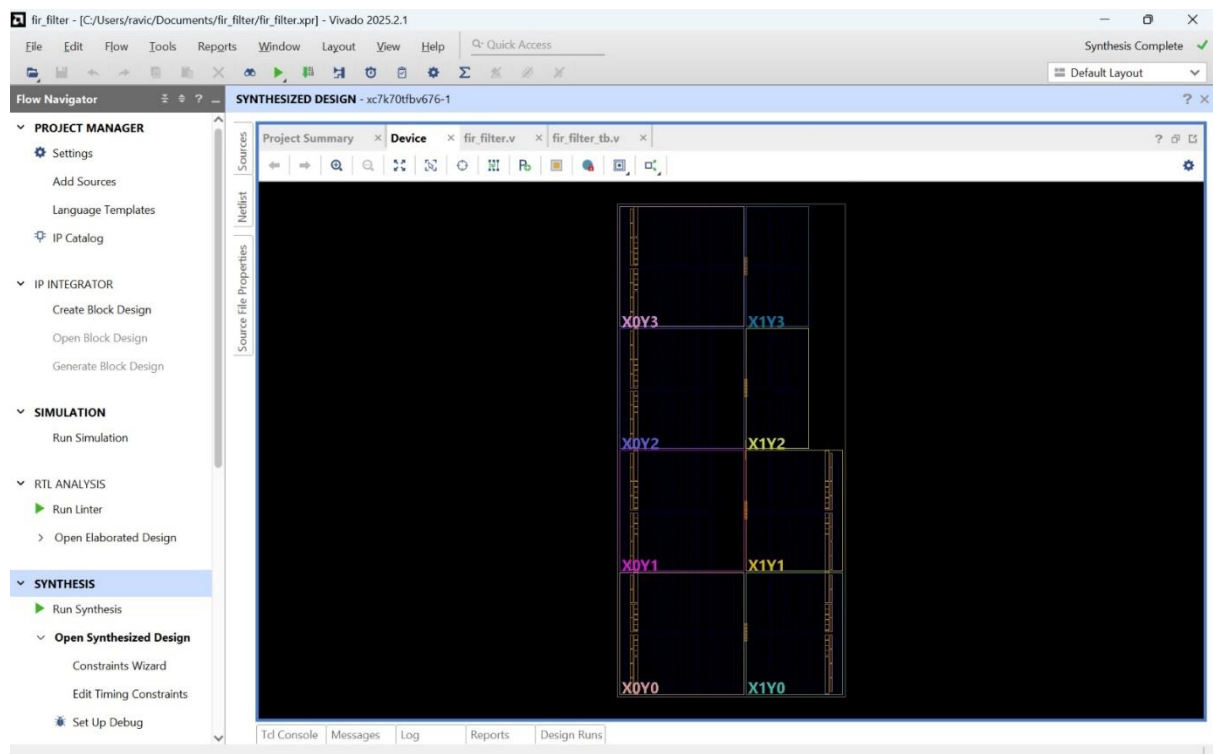
- Clock toggled correctly
- Input values shifted
- FIR output generated
- Output decayed after input returned to zero

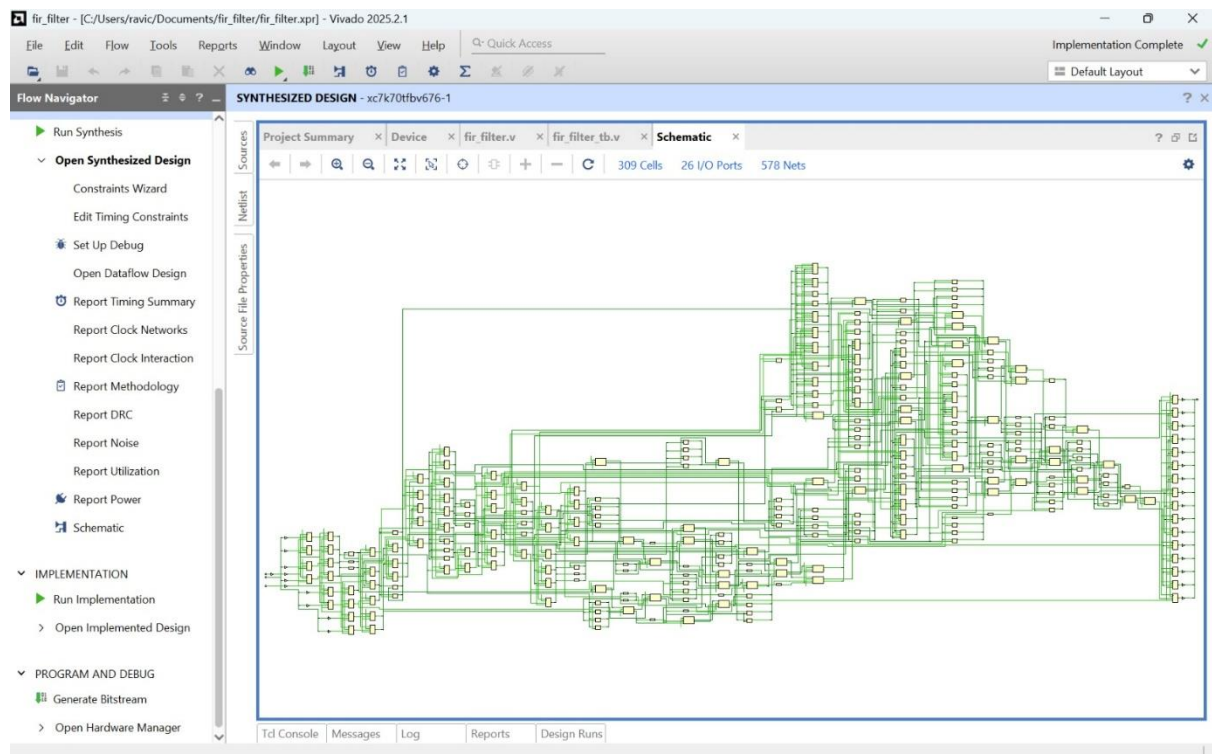
Result:

Functional simulation was successful.



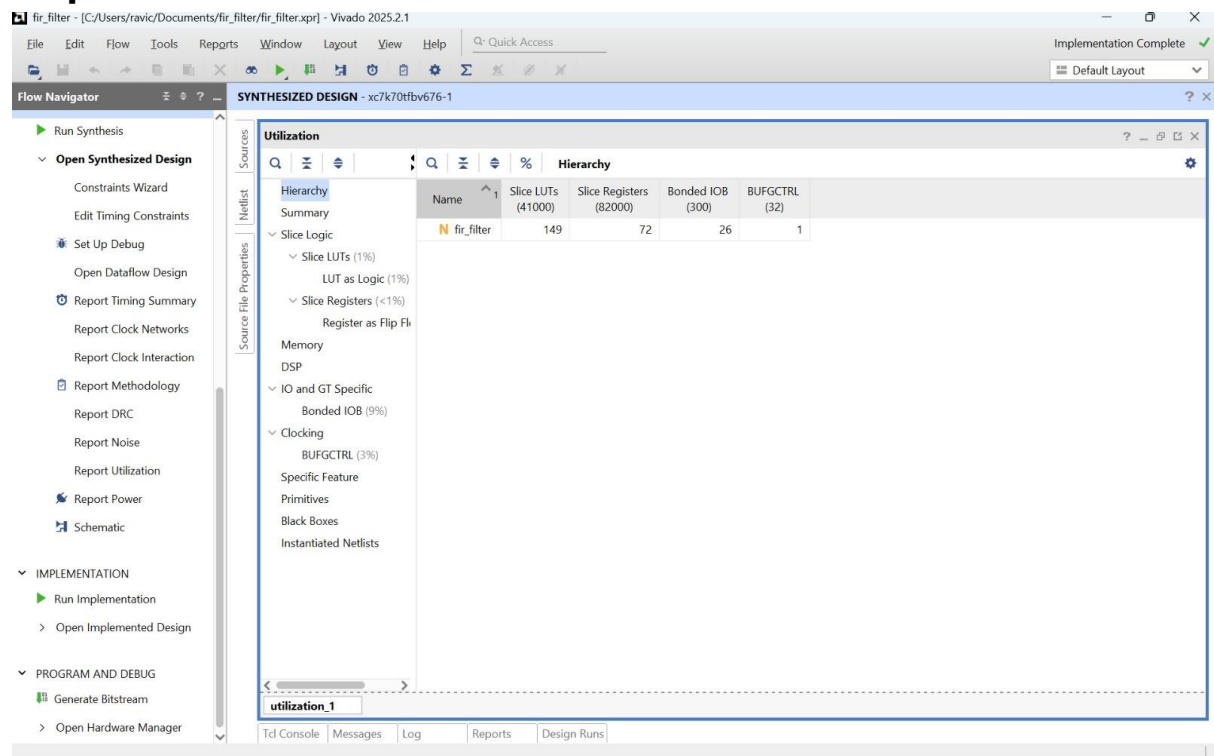
10. Synthesis Design Screenshot:



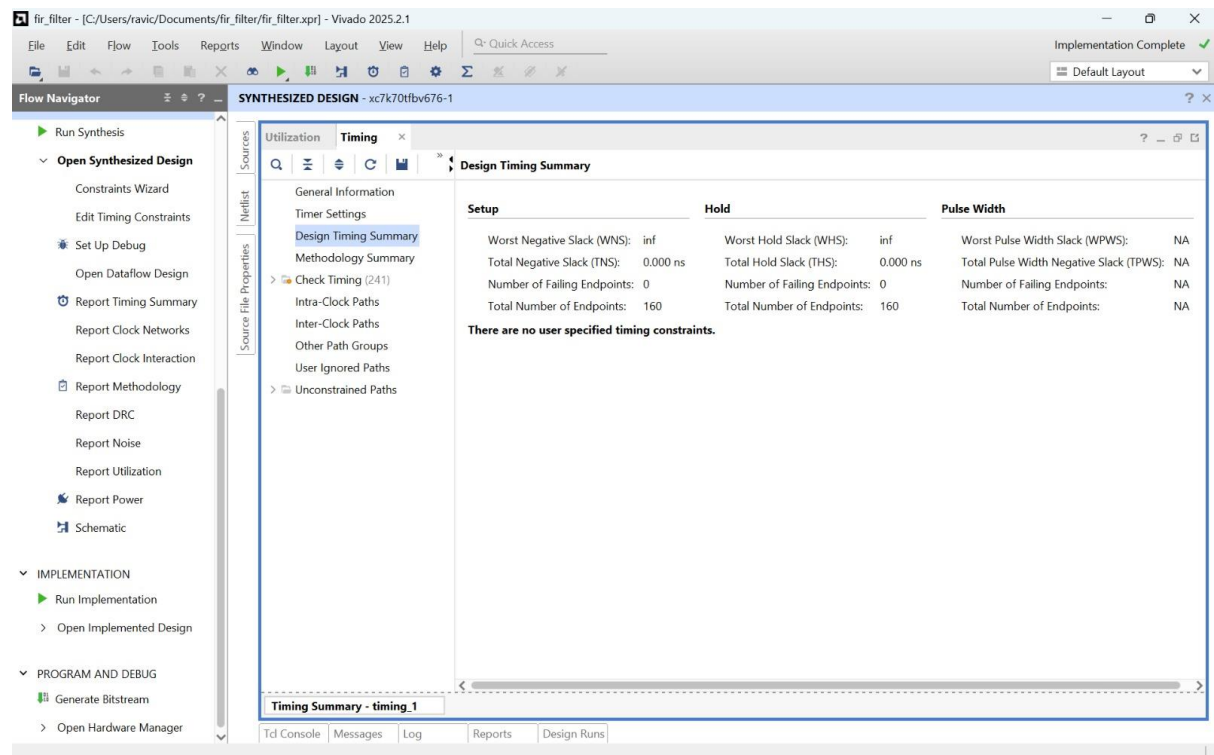


11. Synthesis Results:

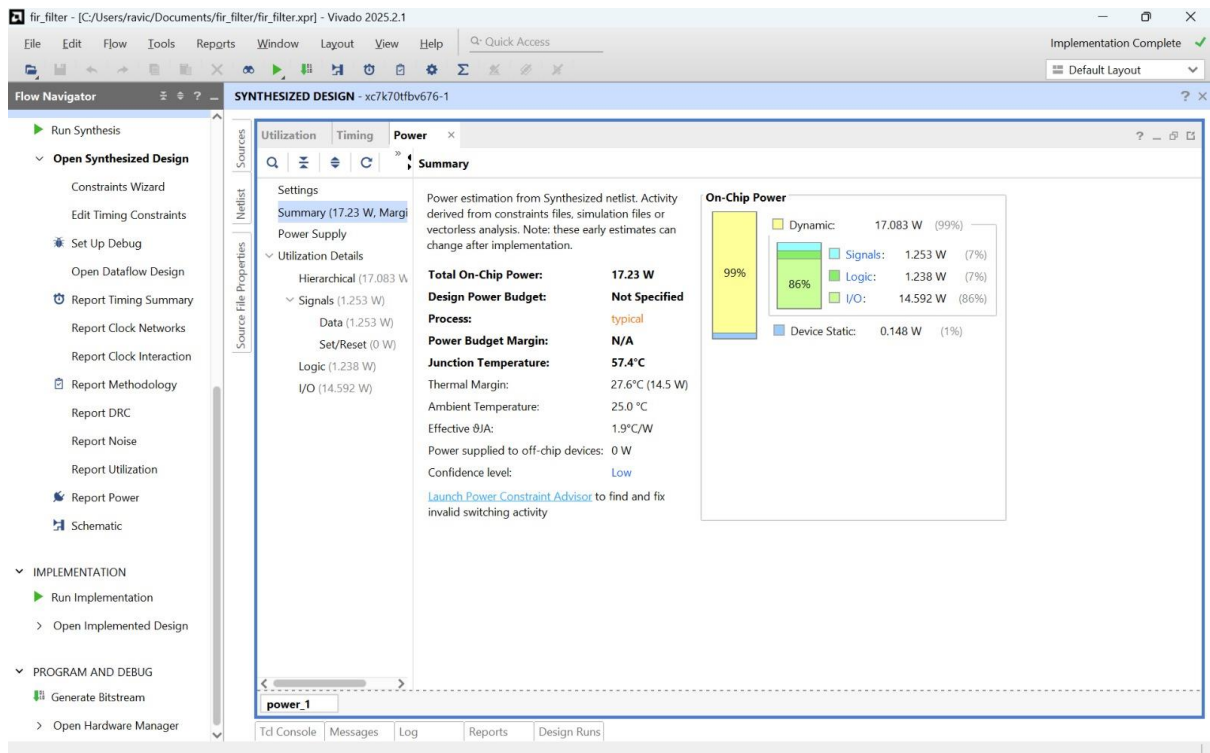
Area results from a screenshot of Vivado Utilization Report:



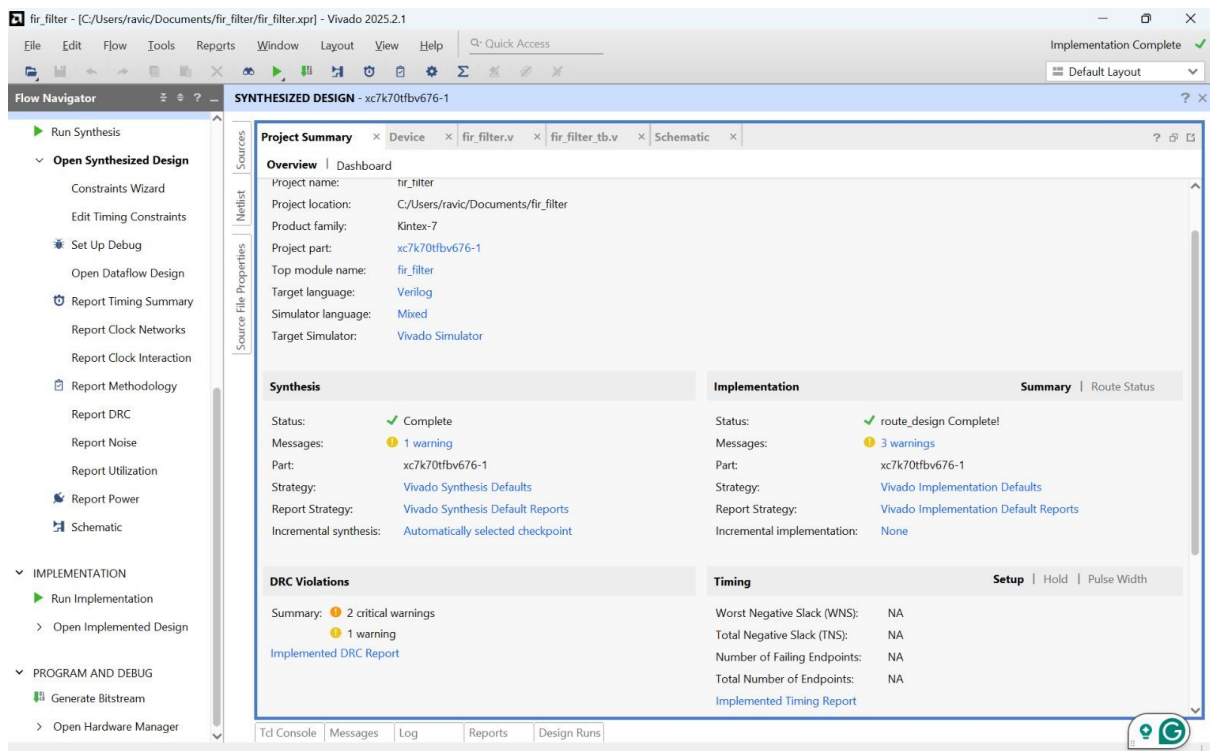
Speed results from a screenshot of Vivado report:

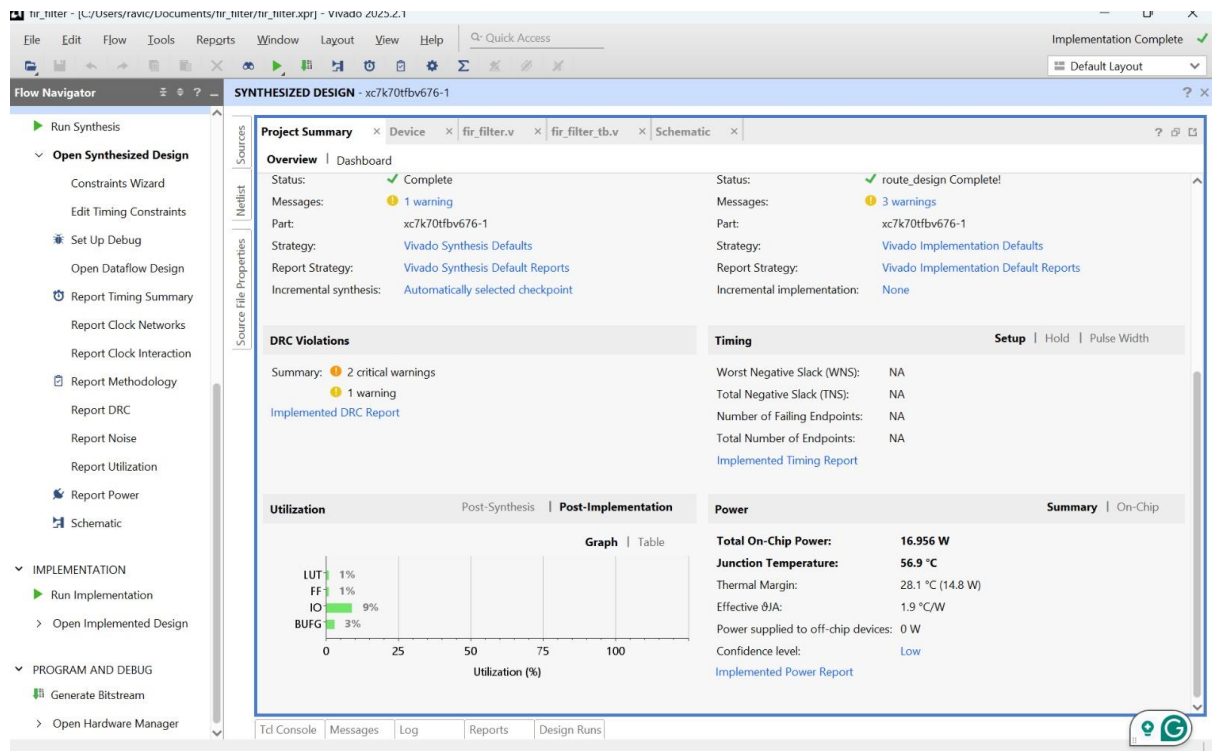


Power results from a screenshot of Vivado report:



12. Summary Report:





13.. Applications

- Audio filtering
- Speech processing
- Radar systems
- ECG filtering
- Communication receivers

14. Advantages

- Stable operation
- Efficient hardware design
- High-speed processing
- Easy implementation in FPGA/VLSI

15. Conclusion

The FIR filter for DSP applications was successfully designed using GNU Octave and implemented in Verilog HDL using Xilinx Vivado. Simulation verified the correct operation of the filter. Synthesis results showed acceptable area, speed, and power performance. The design proves that FIR filters are suitable for VLSI implementation in DSP systems.

16. References

1. Xilinx Vivado User Guide
2. GNU Octave Documentation
3. Digital Signal Processing – Proakis
4. VLSI Design Fundamentals